

Proyecto: Gestor de Procesos (Mini PC)

Curso: IC-6600 Principios de Sistemas Operativos

Profesor: Ing. Cristian Campos Agüero

Estudiante: Keyner Cerdas Morales, carné 2024108270

Repositorio: <https://github.com/KeynerCM/proyecto1Principios>

Documentación Externa

1. Descripción general

El programa simula una minicomputadora (Mini PC) con CPU, memoria principal, disco, pantalla y teclado, y un sistema operativo que administra la ejecución de varios programas escritos en un mini ensamblador (.asm).

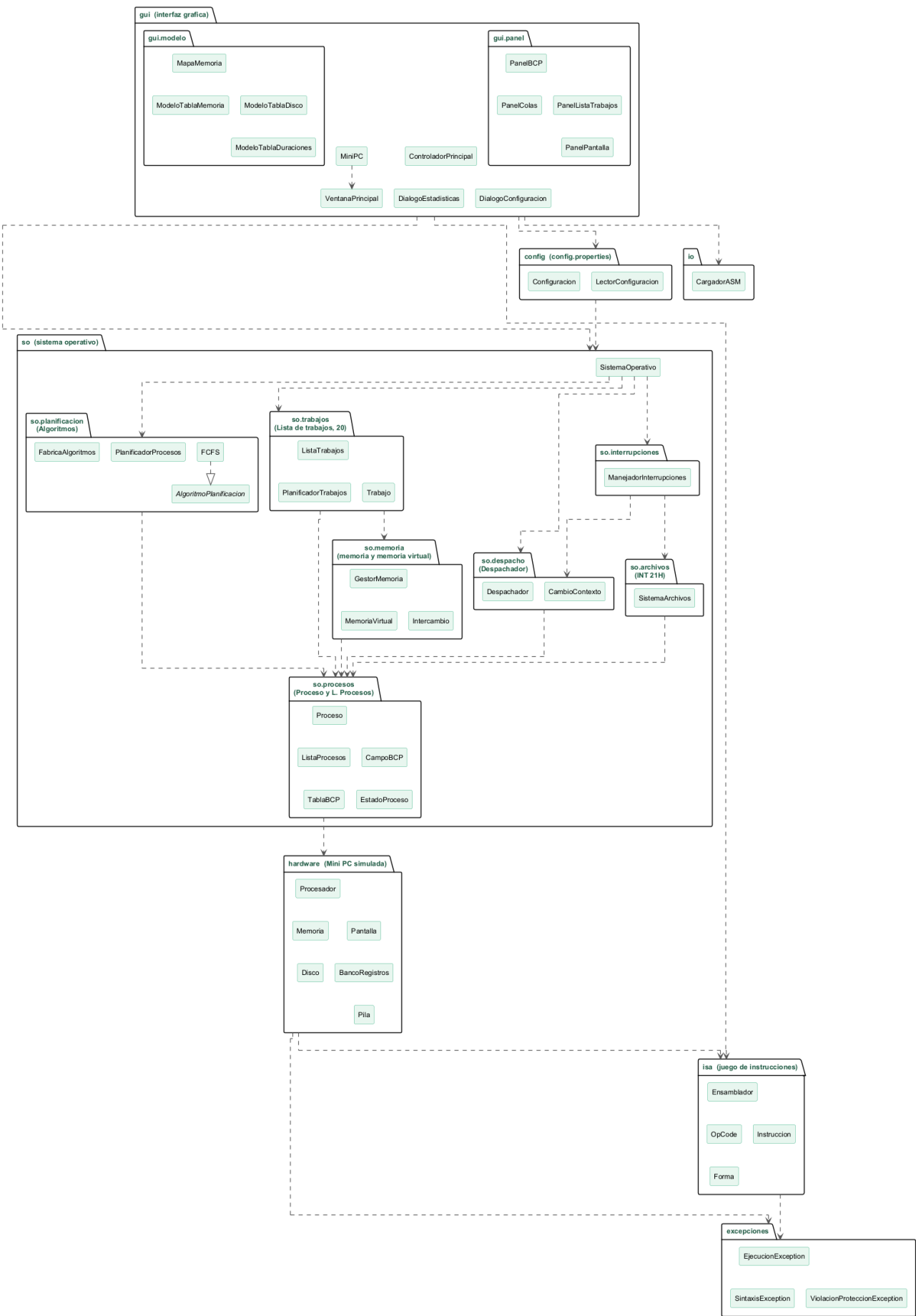
Al cargar uno o varios archivos, el sistema operativo:

1. valida cada línea con expresiones regulares;
2. guarda los programas en el disco, con su entrada en el índice;
3. los pone en la lista de trabajos
4. los admite como procesos, cada uno con su BCP en la memoria del kernel;
5. los ejecuta con planificación FCFS, a través del despachador y el cambio de contexto.

Cada instrucción dura tantos segundos de CPU como su peso. La ejecución avanza de a un segundo con Siguiente o de forma automática con Ejecutar. Al final se muestran las estadísticas de cada proceso.

2. Diagrama de paquetes

Mini PC - Gestor de Procesos: diagrama de paquetes (com.mycompany.minipc)



3. Explicación de cada paquete

3.1 gui: interfaz gráfica

Muestra todo y recibe las acciones del usuario; no toma decisiones del sistema operativo.

- MiniPC arranca el programa y abre VentanaPrincipal.
- VentanaPrincipal es la vista. ControladorPrincipal recibe los botones (Cargar, Ejecutar, Siguiente, Pausar, Reiniciar, Limpiar, Estadísticas, Configuración) y le pide al sistema operativo un segundo de CPU por cada paso. Se comunican por una interfaz (VistaPrincipal), lo que permite probar el controlador sin ventana.
- gui.panel: lista de trabajos, colas de procesos, BCP actual y pantalla con teclado.
- gui.modelo: modelos de las tablas de memoria, disco y estadísticas. Leen directamente la memoria y el disco, sin copiar datos.
- DialogoEstadisticas y DialogoConfiguracion.

Depende de: so (para ejecutar y mostrar el estado), config (para leer y guardar la configuración), io (para leer los .asm) e isa (para validarlos).

3.2 config: configuración externa

LectorConfiguracion lee y escribe config.properties; Configuracion guarda los valores y los valida. Los tamaños no quedan en el código: memoria, porcentaje del kernel, disco, memoria virtual, velocidad y algoritmo.

Depende de: so, porque valida con las mismas reglas que usa el sistema operativo.

3.3 io: lectura de archivos

CargadorASM lee el archivo .asm del disco de la computadora y exige la extensión .asm.

3.4 so: sistema operativo

SistemaOperativo es la fachada: crea y une todas las piezas, y su método tick() es un segundo de CPU:

1. admitir trabajos;
2. si la CPU está libre, elegir el siguiente proceso con FCFS y despacharlo;
3. avanzar el reloj y ejecutar un segundo de CPU;
4. atender la llamada al sistema, si la hubo;
5. terminar el proceso si acabó o tuvo un error.

Depende de sus subpaquetes:

Subpaquete	Responsabilidad	Depende de
so.trabajos	Lista de trabajos (hasta 20) y planificador de trabajos (largo plazo): decide qué programa entra y a qué memoria	so.memoria, so.procesos
so.planificacion	Planificador de procesos (corto plazo) con el algoritmo intercambiable (patrón Estrategia); hoy FCFS	so.procesos
so.despacho	Despachador y cambio de contexto	so.procesos, hardware
so.interrupciones	Llamadas al sistema (INT) y la interrupción de E/S del teclado	so.despacho, so.archivos
so.archivos	INT 21H: crear, abrir, leer, escribir y eliminar archivos de datos	so.procesos, hardware
so.memoria	Memoria principal (primer ajuste), memoria virtual e intercambio (mediano plazo)	so.procesos, hardware
so.procesos	Tabla de BCP en el kernel, Proceso, lista de procesos y los 7 estados	hardware

so.procesos está en la base del sistema operativo porque casi todos los demás subpaquetes trabajan con procesos.

3.5 hardware: la Mini PC

- Procesador: ciclo de instrucción con los registros PC, IR, AC, AX, BX, CX, DX, la bandera de cero, base y alcance.
- Memoria y Disco: arreglos de celdas de texto.
- Pantalla, BancoRegistros (AH y AL como mitades de AX) y Pila (5 posiciones).

No conoce procesos, BCP ni estados: el sistema operativo le carga un contexto y se lo lleva después.

Depende de: isa, para decodificar la instrucción que trae de memoria, y excepciones, para informar los errores de ejecución.

3.6 isa: juego de instrucciones

OpCode define las 16 operaciones con su peso; Forma define cómo se escribe cada una; Interrupcion define los códigos de INT. Ensamblador valida cada línea con una expresión regular que debe coincidir con la línea completa, con exactamente una coma entre operandos, y reporta todos los errores con su número de línea.

Depende de: excepciones.

3.7 excepciones

Errores propios del simulador:

- SyntaxException: errores del archivo al cargarlo;

- `EjecucionException`: un proceso falla al ejecutarse;
- `DesbordamientoException`: la pila se llena o un salto sale del programa;
- `ViolacionProteccionException`: un proceso intenta salir de su región o leer el kernel.

4. Diseño del sistema operativo

4.1 Memoria principal y cálculo del kernel

La memoria es un arreglo de 256 celdas de texto (configurable). El kernel es un **porcentaje de la memoria** (`memoria.kernelPorcentaje` en `config.properties`, por defecto 50 %), y de su tamaño se calcula cuántos BCP caben:

$K = \text{memoria} \times \text{porcentaje} / 100$

$P = (K - C) / B$, como máximo 5

$C = 3$ celdas de cabecera: proceso en ejecución, inicio de la lista de procesos, procesos admitidos

$B = 25$ celdas por BCP

Con 256 y 50 %: $K = 128$, $P = (128 - 3) / 25 = 5$

Direcciones	Contenido
0 a 2	Cabecera del sistema operativo
3 a 27, 28 a 52, 53 a 77, 78 a 102, 103 a 127	Las 5 ranuras de BCP
128 a 255	Zona de usuario: los programas

Por qué 50 %: es el menor porcentaje en el que caben los 5 BCP con la memoria por defecto, así deja el mayor espacio posible para los programas.

Porcentaje	Kernel (256)	BCP que caben	Zona de usuario
20 %	51	1	205
25 %	64	2	192
40 %	102	3	154
50 %	128	5	128
60 %	153	5 (25 celdas sin usar)	103

Con un porcentaje menor caben menos procesos a la vez; con uno mayor sobran celdas en el kernel. El porcentaje debe estar entre 10 y 90, dejar lugar para al menos un BCP y dejar al menos 32 celdas para los programas.

4.2 El BCP vive en la memoria

La información de cada proceso no está en objetos de Java: el BCP ocupa 25 celdas consecutivas del kernel. La clase Proceso solo guarda la dirección de su BCP, y cada consulta (getPc(), getEstado(), ...) lee la celda correspondiente.

Campos del BCP, en orden:

- PID, programa, estado y prioridad;
- PC, IR, AC, AX, BX, CX, DX y la bandera de cero;
- SP y **Pila[1] a Pila[5], cada elemento en su propia celda;**
- base y alcance;
- CPU, tiempo de inicio y tiempo empleado;
- archivos abiertos y siguiente BCP.

4.3 Lista de procesos enlazada

La cabecera guarda la dirección del primer BCP y cada BCP guarda en el campo SIGUIENTE la dirección del próximo. Las colas que muestra la interfaz (en CPU, preparados, en espera, suspendidos) se obtienen recorriendo esa lista y filtrando por estado.

4.4 Estados del proceso

NUEVO, PREPARADO, EJECUCION, EN_ESPERA, SUSPENDIDO_PREPARADO, SUSPENDIDO_EN_ESPERA y FINALIZADO.

Transición	Cuándo ocurre
NUEVO -> PREPARADO	El planificador de trabajos lo admite y su programa cabe en la memoria principal
NUEVO -> SUSPENDIDO_PREPARADO	Lo admite, pero su programa no cabe: va a la memoria virtual
SUSPENDIDO_PREPARADO -> PREPARADO	Se libera memoria y el intercambio lo trae
PREPARADO -> EJECUCION	FCFS lo elige y el despachador le da la CPU
EJECUCION -> EN_ESPERA	Ejecuta INT 09H
EN_ESPERA -> PREPARADO	Llega el ENTER del teclado (interrupción de E/S)
EJECUCION -> FINALIZADO	INT 20H, fin del programa o un error

4.5 Planificación FCFS modular

PlanificadorProcesos delega la elección en una AlgoritmoPlanificacion (patrón Estrategia). FCFS devuelve el primer proceso PREPARADO de la lista, que es el que llegó primero; no es expropiativo. El algoritmo se elige en la configuración mediante FabricaAlgoritmos. Otro

algoritmo (SPN, SRT, RR o HRRN) se agrega como una clase nueva que implementa la interfaz, sin cambiar el despachador ni el resto del sistema.

4.6 Despachador y cambio de contexto

Al sacar un proceso de la CPU, el despachador:

1. guarda el contexto de la CPU en su BCP: PC, IR, AC, registros, bandera de cero y pila;
2. actualiza su estado;
3. deja la CPU libre.

Al darle la CPU a otro proceso:

1. actualiza su BCP a EJECUCION;
2. carga en la CPU sus registros base y alcance;
3. restaura su contexto desde el BCP.

Cada paso se escribe en la consola del sistema operativo.

4.7 CPU, pesos y reloj simulado

Cada vez que se presiona **Siguiente** pasa un segundo de CPU. Una instrucción de peso n dura n segundos:

- en el primero se hace el fetch: el texto de la celda apuntada por el PC pasa al IR y el PC avanza;
- en el último se aplica su efecto.

El reloj es simulado: empieza en 00:00:00 y avanza un segundo por cada segundo de CPU. Así las estadísticas no dependen de la velocidad del usuario.

4.8 Interrupciones, teclado y pantalla

Instrucción	Peso	Qué hace el sistema operativo
INT 20H	2	Termina el proceso y libera su memoria y su BCP
INT 10H	2	Imprime DX en la pantalla, con el proceso como prefijo: [P2] 14
INT 09H	variable	El proceso pasa a EN_ESPERA y la pantalla muestra [P1] >> Ingresar valor:. La CPU espera a ese proceso: no pasa a otro mientras tanto, y los procesos se ejecutan de uno en uno en orden de llegada. El ENTER (0 a 255, validado con regex) pone el valor en el DX del BCP y el proceso vuelve a PREPARADO sin perder su lugar
INT 21H	5	Archivos: AH = 3Ch crear, 3Dh abrir, 4Dh leer, 40h escribir, 41h eliminar; DX apunta al nombre y AL lleva el byte

4.9 Disco e índice

El disco es un arreglo de 512 celdas de texto (configurable):

- las primeras 20 celdas son el **índice**, con entradas nombre|inicio|tamaño;
- luego está el área de archivos (programas y archivos de datos), donde los programas se guardan contiguos con primer ajuste;
- al final están las 64 celdas de **memoria virtual**.

4.10 Memoria virtual

- **NUEVO -> SUSPENDIDO_PREPARADO**: si un programa admitido no cabe en la memoria principal, su imagen se copia al área de memoria virtual del disco.
- **SUSPENDIDO_PREPARADO -> PREPARADO**: cuando un proceso termina y libera memoria, el intercambio trae primero a los suspendidos, en orden de llegada, antes de admitir trabajos nuevos.
- Si tampoco cabe en la memoria virtual, el trabajo sigue en la lista de trabajos hasta que se libere espacio.
- El BCP nunca sale del kernel; solo viaja la imagen del programa. Al moverla se reubican la base y el PC del proceso.
- Suspende un proceso en espera (EN_ESPERA -> SUSPENDIDO_EN_ESPERA) está implementado, pero no se usa durante la ejecución: como la CPU espera al proceso que pidió el teclado, sacarlo de la memoria no le daría lugar a nadie que pudiera ejecutarse.

5. Protección y seguridad: estrategia

1. **Registros base y alcance**. Antes de cada fetch la CPU compara el PC con la región del proceso, y cada salto (JMP, JE, JNE) se valida antes de aplicarse. Salir de la región termina el proceso con "Violación de protección" o "Desbordamiento", y los demás siguen.
2. **Kernel inaccesible**. Los procesos leen la memoria con una operación que rechaza cualquier dirección del kernel. Solo el sistema operativo escribe los BCP.
3. **DX relativo a la base**. MOV DX, "archivo.txt" guarda en DX la posición del nombre dentro del programa (como DS:DX en x86). INT 21H compara DX con el alcance antes de sumar la base, así un proceso no puede leer datos de otro.
4. **Programas protegidos**. INT 21H con AH = 41h solo puede eliminar archivos de datos; los programas .asm del disco no se pueden borrar.
5. **Memoria virtual separada**. El intercambio solo puede escribir dentro del área de memoria virtual del disco.

6. **Validación de entradas.** Los archivos se validan línea por línea con expresiones regulares de la línea completa; el teclado solo acepta números de 0 a 255; la configuración se valida antes de aplicarse.
7. **Errores aislados.** Un error de ejecución (pila llena o vacía, salto fuera del programa, instrucción inválida) termina solo al proceso que lo produjo y queda registrado en la consola y en las estadísticas.

6. Estadísticas

Al terminar el último proceso se abre la ventana de estadísticas. Por cada proceso muestra:

- **hora de inicio, hora final y duración en segundos**, con el reloj simulado;
- tiempo de CPU y espera;
- resultado.

También muestra el uso de la CPU, la ocupación máxima de la memoria principal y de la memoria virtual, y los contadores de instrucciones ejecutadas y accesos a memoria.

7. Decisiones de diseño

Decisión	Motivo
Capas con dependencias que solo bajan	Cada parte se entiende, se prueba y se cambia por separado
Fachada SistemaOperativo	La interfaz habla con un solo objeto y no conoce los detalles internos
Patrón Estrategia para la planificación	Agregar un algoritmo no cambia el resto del sistema
Memoria y disco como arreglos de texto	Lo que se ve en las tablas es exactamente lo que hay en memoria, incluidos los BCP
Kernel por porcentaje y BCP calculados	El tamaño del kernel no está fijo en el código y se adapta a la memoria configurada
Proceso solo con la dirección de su BCP	Toda la información del proceso vive en la memoria, no en objetos de Java
Reloj simulado por peso	Los tiempos no dependen de qué tan rápido se use la interfaz
Configuración en config.properties	Los tamaños y la velocidad se cambian sin tocar el código